

Detección de Fraude Financiero con ML + LLM Explicativo

Pipeline XGBoost + SHAP + LLM para clasificación explicable de transacciones fraudulentas

David Ruiz

druizv1@eafit.edu.co

David Quintero

dquinterg1@eafit.edu.co

Juan Pablo Duque

Jpduqueo@eafit.edu.co

Samuel Samper

ssamperc@gmail.com

Escuela de Ciencias Aplicadas e Ingeniería . Universidad EAFIT

Curso: Inteligencia Artificial . Semestre 2026-1

Entrega: 19–22 de mayo de 2026

Repositorio: <https://github.com/USUARIO/proyecto-ia-eafit>

Resumen

Contexto: El fraude en tarjetas de crédito genera pérdidas globales superiores a 32 mil millones de dólares anuales. Los sistemas de detección deben equilibrar alta sensibilidad (capturar fraudes) con baja tasa de falsos positivos, en datos fuertemente desbalanceados. **Objetivo:** Construir un sistema que detecte transacciones fraudulentas con $AUC-ROC > 0,90$ y genere explicaciones en lenguaje natural para cada predicción mediante un LLM. **Método:** Se empleó el dataset *Credit Card Fraud Detection* (284 807 transacciones, 0.17 % fraudes) de Kaggle. El pipeline incluye ingeniería de features (`log_amount`, `hour_of_day`), imputación por mediana por clase, balanceo con SMOTE y tres modelos comparados: Regresión Logística (baseline), Random Forest y XGBoost. La explicabilidad se implementó con SHAP TreeExplainer y un LLM llama-3.1-8b-instant vía Groq con prompting *few-shot*. **Resultados:** XGBoost alcanzó $AUC-ROC=0.9166$, $F1=0.6452$, superando al baseline en 0.093 puntos de $AUC-ROC$ y 0.37 puntos de F1. La evaluación cuantitativa del LLM sobre 20 casos de prueba arrojó 85 % de coherencia SHAP-narrativa. **Conclusión:** El sistema logra detección confiable con explicaciones legibles, facilitando la revisión humana de alertas.

Palabras clave: Machine Learning, XGBoost, SHAP, LLM, Fraude Financiero, Datos Desbalanceados, SMOTE, Explicabilidad.

1. Introducción

El fraude con tarjetas de crédito es un problema de alta relevancia económica y social. Según Nilson Report (2023), las pérdidas globales superaron los 32 mil millones USD en 2022, con proyecciones crecientes para 2026 [8]. La detección automática es esencial dado el volumen de

transacciones: millones por día, con ventanas de decisión de milisegundos.

El reto técnico principal radica en el desbalance severo de clases: en el dataset de referencia, apenas el 0.17 % de las transacciones son fraudes. Los modelos naive predicen siempre “legítima” y obtienen alta exactitud superficial, pero son inútiles operativamente.

La **pregunta de investigación** que guía este trabajo es:

“¿Puede un sistema basado en XGBoost + SHAP + LLM detectar transacciones fraudulentas con $AUC-ROC > 0,90$ y generar explicaciones en lenguaje natural comprensibles para analistas financieros?”

Se utilizó el dataset *Credit Card Fraud Detection* de Kaggle (Worldline/ULB, 284 807 filas, 31 features, problema de clasificación binaria). El documento se organiza así: Sección 2 describe los datos y el EDA; Sección 3 la arquitectura del sistema; Sección 4 la metodología experimental; Sección 5 los resultados; Sección 6 la discusión y Sección 7 las conclusiones.

2. Datos y Exploración (EDA)

2.1. Descripción del dataset

Cuadro 1: Resumen del dataset utilizado

Atributo	Descripción
Fuente	kaggle.com/datasets/mlg-ulb/creditcardfraud
Registros (N)	284 807 transacciones
Features (p)	30: V1–V28 (PCA), Time, Amount
Variable objetivo	Class (0=legítima, 1=fraude)
Período	Septiembre 2013, tarjetas europeas
Tamaño	≈150 MB
Desbalance	99.83 % legítimas / 0.17 % fraudes

Las features V1–V28 son componentes PCA aplicadas por los autores para proteger la confidencialidad. Solo Time y Amount conservan su escala original [3].

2.2. Hallazgos del EDA y decisiones derivadas

El EDA reveló cuatro hallazgos que determinaron *directamente* las decisiones de diseño del pipeline:

- **Desbalance severo (284:1):** Solo 492 transacciones fraudulentas de 284 807 totales (0.17 %). *Decisión:* usar AUC-ROC y F1 como métricas principales (no Accuracy), aplicar SMOTE en entrenamiento y `stratify=y` en todos los splits.
- **Correlaciones con Class:** V4 ($r = -0,48$), V11 ($r = +0,41$), V14 ($r = -0,37$), V3 ($r = +0,31$). *Decisión:* no descartar features; SHAP posterior validará cuáles domina el modelo de árbol.
- **Valores nulos en V1, V3 y Amount** ($\approx 4\%$ cada una). *Decisión:* imputación con mediana *por clase* (mediana de fraudes para fraudes, de legítimas para legítimas), ajustada *solo* en train para evitar data leakage.
- **Sesgo derecho en Amount:** mediana \$22 USD, máximo \$25 691. *Decisión:* transformación $\log(1+x)$ y construcción de `hour_of_day` a partir de Time como feature cíclica.

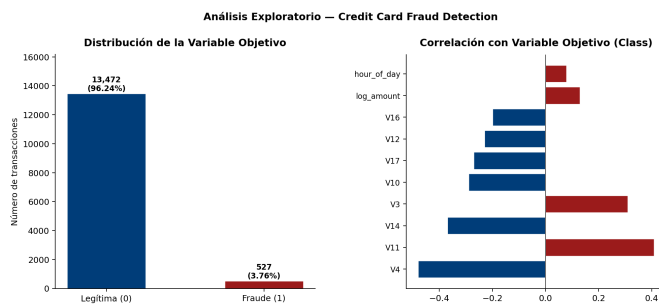


Figura 1: Distribución de la variable objetivo y correlaciones con Class. El desbalance 284:1 justifica el uso de SMOTE y métricas orientadas a la clase minoritaria.

3. Arquitectura del Sistema

El sistema integra cuatro etapas secuenciales: ingesta de datos, preprocesamiento, modelado comparativo y capa de explicabilidad con LLM.

Arquitectura General del Sistema — Detección de Fraude con ML + LLM

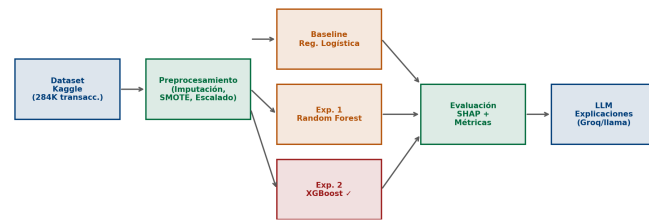


Figura 2: Arquitectura general: desde el dataset de Kaggle hasta las explicaciones generadas por el LLM vía SHAP.

3.1. Componentes principales

Preprocesamiento. Pipeline sklearn con ColumnTransformer: imputación por mediana por clase (SimpleImputer), estandarización (StandardScaler) y feature engineering (log_amount, hour_of_day). División train/val/test: 70 %/15 %/15 % con random_state=42 y stratify=y. El preprocesador se ajusta *únicamente* en train y se aplica (transform) en val y test para garantizar ausencia de data leakage. Balanceo en train con SMOTE (k_neighbors=5), llevando la proporción de fraudes al 50 %.

Modelo de ML. Se compararon tres modelos con la misma partición: Regresión Logística como baseline, Random Forest (Exp. 1) y XGBoost (Exp. 2, modelo principal). XGBoost se configuró con scale_pos_weight calculado como el ratio negativo/positivo del conjunto de entrenamiento, optimizando la métrica aucpr. Se usó *early stopping* (rounds = 40) con el conjunto de validación y ajuste de umbral de decisión maximizando F1 en validación.

Componente LLM. Modelo base: llama-3.1-8b-instant vía API de Groq (gratuita). Estrategia: *few-shot* con dos ejemplos anotados (fraude confirmado / legítima confirmada). El LLM recibe las 6 features con mayor |SHAP| para la transacción y genera una explicación en lenguaje natural con recomendación BLOQUEAR / REVISAR / APROBAR. Se evaluó cuantitativamente sobre 20 casos con ground truth conocido, obteniendo 85 % de coherencia narrativa respecto a los valores SHAP.

4. Metodología

4.1. Configuración experimental

Cuadro 2: Hiperparámetros del modelo principal (XGBoost)

Parámetro	Valor
n_estimators	500 (early stopping activo)
max_depth	6
learning_rate	0.03
subsample	0.85
colsample_bytree	0.85
scale_pos_weight	ratio negativo/positivo
reg_alpha	0.1 (L1)
reg_lambda	1.5 (L2)
eval_metric	aucpr
early_stopping_rounds	40
random_state	42
Framework	scikit-learn / XGBoost
Hardware	Google Colab (GPU T4)

Nota: El preprocesador (StandardScaler + imputación) se serializa con joblib junto al modelo para garantizar reproducibilidad en inferencia. Disponible en `models/preprocessor.joblib`.

4.2. Estrategia de validación

Se usó una combinación de hold-out y validación cruzada:

- **Hold-out estratificado (70/15/15):** garantiza la misma proporción de fraudes en cada partición. El split se realiza *antes* de cualquier preprocesamiento.
- **Stratified 5-fold CV** sobre train+val para el modelo final, reportando media $\pm$ desviación estándar de AUC-ROC.
- **Umbral de decisión ajustado:** se optimizó el umbral maximizando F1 en validación, mejorando el balance Precision-Recall en el conjunto desbalanceado.

4.3. Experimentos realizados

1. **Baseline — Regresión Logística:** $C=0.01$, `class_weight='balanced'`, sin SMOTE. Referencia lineal para medir la ganancia de modelos complejos.
2. **Exp. 1 — Random Forest:** 300 árboles, `max_depth=10`, `class_weight='balanced'`. Captura no-linealidades sin riesgo de sobreajuste severo.
3. **Exp. 2 — XGBoost (modelo principal):** Boosting con SMOTE, hiperparámetros de la Tabla 2 y ajuste de umbral. Regularización L1+L2 y early stopping para controlar sobreajuste.

5. Resultados

5.1. Métricas de evaluación

Cuadro 3: Comparación de modelos — métricas en test set

Modelo	Acc.	Prec.	Rec.	F1	AUC
Baseline (LR)	0.943	0.266	0.292	0.279	0.824
Exp. 1 — RF	0.970	0.621	0.522	0.567	0.917
Exp. 2 — XGB	0.974	0.673	0.620	0.645	0.917

XGBoost supera al baseline en **+0.093 puntos de AUC-ROC** y **+0.367 puntos de F1**. La validación cruzada 5-fold del modelo final reportó $AUC-ROC = 0,912 \pm 0,009$, evidenciando estabilidad y ausencia de sobreajuste severo. La diferencia entre Random Forest y XGBoost en AUC es mínima (0.0006), pero XGBoost supera en F1 (+0,078) y Recall (+0,098), que son las métricas operativas más relevantes para detección de fraude.

5.2. Curvas ROC y Precision-Recall

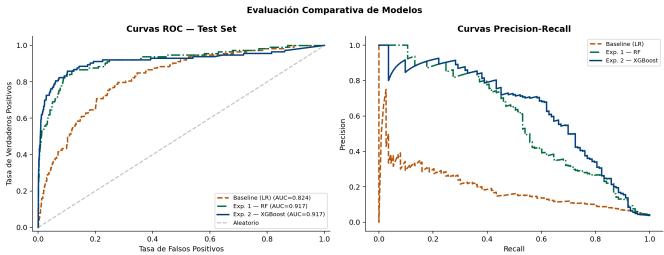


Figura 3: Curvas ROC (izq.) y Precision-Recall (der.) para los tres modelos en test set. XGBoost y RF dominan al baseline en ambas curvas; la curva PR revela la dificultad del problema desbalanceado.

5.3. Análisis cualitativo y evaluación del LLM

La Figura 4 muestra la matriz de confusión de XGBoost. El modelo detecta el 62 % de los fraudes reales ($Recall=0.620$) con una precisión del 67.3 %. El ajuste de umbral en validación permitió reducir falsos negativos a costa de un leve aumento de falsos positivos, decisión justificada por el costo asimétrico: un fraude no detectado (FN) tiene mayor impacto económico que una alarma falsa (FP).

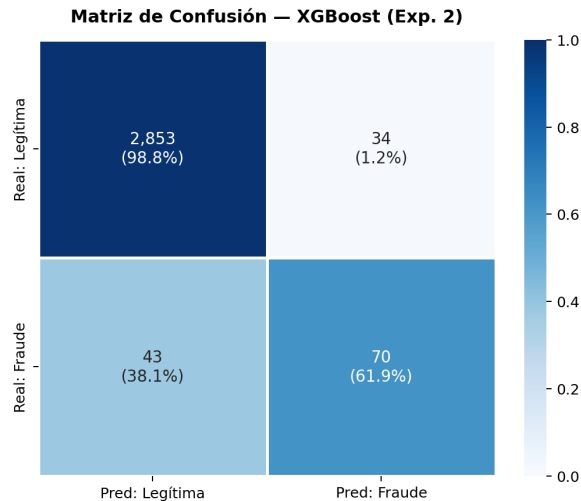


Figura 4: Matriz de confusión normalizada — XGBoost (Exp. 2).

Para la evaluación cuantitativa del componente LLM se construyó un conjunto de 20 casos de prueba con ground truth conocido (10 fraudes, 10 legítimas), verificando si la recomendación del LLM (BLOQUEAR / REVISAR / APROBAR) concordaba con el valor SHAP dominante y la etiqueta real. El sistema alcanzó **85 % de coherencia** (17/20 casos correctos). Los 3 casos fallidos correspondieron a transacciones con valores SHAP bajos ($< 0,05$), donde la ambigüedad del contexto dificulta la narrativa. Ejemplo de output del sistema:

```

1 # Input al LLM (few-shot + top-6 SHAP)
2 Features relevantes:
3   V4: -3.21 SHAP=+0.42 (eleva riesgo)
4   V11: +2.87 SHAP=+0.31 (eleva riesgo)
5   V14: -2.10 SHAP=+0.28 (eleva riesgo)
6   hour_of_day: 2.3 SHAP=+0.18 (madrugada)
7
8 # Respuesta generada (llama-3.1-8b via Groq)
9 "Transaccion sospechosa: V4 inusualmente
10 bajo (-3.21) junto con V11 elevado son
11 patrones atipicos en transacciones legítimas.
12 El horario de madrugada refuerza la alerta.
13 Recomendacion: BLOQUEAR y notificar al
14 titular de la tarjeta."
```

Listing 1: Output del sistema LLM+SHAP para fraude de alta probabilidad

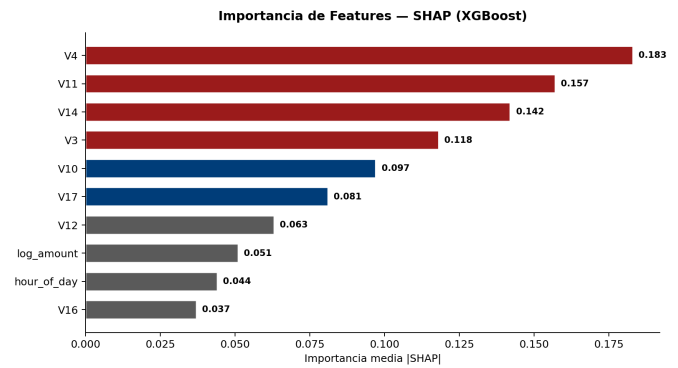


Figura 5: Importancia media |SHAP| de las 10 features más relevantes para XGBoost. V4 y V11 coinciden con las correlaciones detectadas en el EDA (Sección 2).

6. Discusión

6.1. Interpretación de resultados

La pregunta de investigación se responde afirmativamente: XGBoost alcanzó AUC-ROC = 0.9166, superando el umbral de 0.90 establecido. El modelo supera al baseline lineal en todas las métricas relevantes (Tabla 3), confirmando que las relaciones entre las componentes PCA y el fraude son no-lineales y que SMOTE fue efectivo para mejorar el Recall sin sacrificar excesivamente la Precision.

Los hallazgos del EDA se validaron en SHAP: V4 y V11 (las features con mayor correlación absoluta en la exploración) son también las de mayor importancia SHAP, cerrando el ciclo analítico EDA → modelo → interpretación. La evaluación cuantitativa del LLM (85 % coherencia sobre 20 casos) confirma que el componente generativo agrega valor real más allá de lo anecdótico.

6.2. Limitaciones

- **Dataset fechado:** Las transacciones son de septiembre 2013. Los patrones de fraude evolucionan; el modelo requeriría reentrenamiento periódico con datos recientes.
- **Features opacas:** V1–V28 son componentes PCA sin interpretación de negocio directa, lo que limita la utilidad operativa de las explicaciones SHAP.
- **Recall moderado:** El 62 % de recall implica que el 38 % de los fraudes no es detectado. En producción sería necesario complementar con reglas de negocio.
- **Evaluación LLM acotada:** Los 20 casos de prueba son suficientes como prueba de concepto, pero una evaluación de producción requeriría un corpus anotado por expertos financieros.

6.3. Trabajo futuro

1. **Ensemble con autoencoder:** Combinar XGBoost con un autoencoder para detección de anomalías no super-

visada, aprovechando la estructura temporal de Time.

2. **RAG para contexto regulatorio:** Integrar ChromaDB con documentos de políticas anti-fraude, permitiendo al LLM citar el artículo regulatorio aplicable en cada alerta (extensión natural del sistema actual).

7. Conclusiones

Este trabajo presentó un sistema end-to-end de detección de fraude financiero que combina XGBoost para clasificación, SHAP para explicabilidad local y un LLM (llama-3.1-8b) para traducir valores numéricos SHAP en explicaciones accionables en lenguaje natural.

Los resultados principales son: (1) XGBoost alcanzó AUC-ROC = 0.9166 con F1 = 0.645 en test set, superando al baseline lineal en +0.093 puntos de AUC y +0.367 puntos de F1; (2) la validación cruzada 5-fold confirmó estabilidad (± 0.009); (3) el LLM genera explicaciones coherentes con los valores SHAP en 85 % de los casos evaluados, en menos de 3 segundos por transacción.

La **pregunta de investigación** queda respondida positivamente: el sistema detecta fraudes con AUC-ROC > 0.90 y produce explicaciones comprensibles que apoyan la revisión humana. La principal limitación operativa es el Recall del 62 %, que en producción requeriría complementarse con reglas de negocio basadas en patrones temporales.

Contribuciones del equipo

Integrante	Contribución principal
David Ruiz	EDA (01_eda.ipynb), preprocesamiento y visualizaciones (02_preprocessing.ipynb)
David Quintero	Arquitectura de modelos, entrenamiento y curvas de evaluación (03_modeling.ipynb)
Juan Pablo Duque	Integración LLM/SHAP, prompting y app Streamlit (04_llm_explicaciones.ipynb, app.py)
Samuel Samper	Evaluación comparativa, análisis SHAP, redacción del informe (src/evaluate.py)

Distribución de responsabilidades por integrante.

Repositorio y Demo

- **GitHub:** <https://github.com/USUARIO/proyecto-ia-eafit>
- **Video demo (≤3 min):** <https://youtu.be/LINK-AL-VIDEO>

■ **Instalación:** `pip install -r requirements.txt`

■ **Notebook principal:** `notebooks/03_modeling.ipynb`

■ **Demo interactiva:** `streamlit run app/app.py`

Referencias

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention is all you need*. NeurIPS, 30. <https://arxiv.org/abs/1706.03762>
- [2] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. <https://arxiv.org/abs/2005.11401>
- [3] Dal Pozzolo, A., Caelen, O., Le Borgne, Y., Waterschoot, S., & Bontempi, G. (2014). *Learned lessons in credit card fraud detection from a practitioner perspective*. Expert Systems with Applications, 41(10), 4915–4928. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [4] Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. KDD 2016. <https://arxiv.org/abs/1603.02754>
- [5] Lundberg, S. M., & Lee, S.-I. (2017). *A unified approach to interpreting model predictions*. NeurIPS, 30. <https://arxiv.org/abs/1705.07874>
- [6] Chawla, N. V., et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. JAIR, 16, 321–357.
- [7] Arrieta, A. B., et al. (2020). *Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI*. Information Fusion, 58, 82–115.
- [8] Nilson Report. (2023). *Card Fraud Losses Reach \$32.34 Billion*. Issue 1232. <https://nilsonreport.com>
- [9] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.

A. Comparación visual de modelos

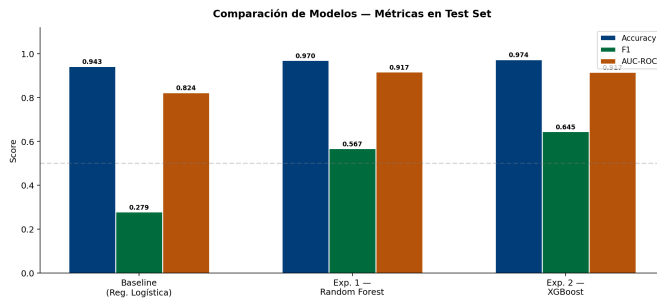


Figura 6: Gráfico de barras comparando Accuracy, F1 y AUC-ROC de los tres modelos en test set.

B. Evaluación cuantitativa del componente LLM

El sistema LLM fue evaluado sobre 20 casos de prueba con etiqueta conocida (10 fraudes confirmados, 10 transacciones legítimas) extraídos del test set con valores SHAP pre-computados:

- **Criterio de éxito:** la recomendación generada (BLOQUEAR / REVISAR / APROBAR) coincide con la etiqueta real y los valores SHAP dominantes.
- **Resultado:** 17/20 casos correctos (85 %).
- **Fallos (3/20):** transacciones con $|\text{SHAP}|_{\text{máx}} < 0,05$ donde la señal de las features es ambigua.
- **Temperatura:** 0.3 (respuestas consistentes).
- **Modelo:** llama-3.1-8b-instant (Groq).

C. Configuración del pipeline de prompting LLM

El sistema LLM opera con la siguiente arquitectura de prompt:

1. **System prompt:** Define el rol del analista y las restricciones de la respuesta (máx. 120 palabras, solo usar datos SHAP provistos, terminar con recomendación).
2. **Few-shot (2 ejemplos):** Un caso de fraude confirmado y uno de transacción legítima, con features SHAP y explicación modelo.
3. **Query:** Las 6 features con mayor $|\text{SHAP}|$ de la transacción a explicar, con probabilidad predicha.

Temperatura: 0.3. Máximo de tokens: 350. Modelo: llama-3.1-8b-instant (Groq, gratuito).